

Project Handoff and Run Guide

F1 Car Vision Intelligence System

A practical guide for a developer with basic Python knowledge who needs to install, run, understand, and modify the project.

GitHub	github.com/MohammedHssan11/f1-car-vision-intelligence-system
Latest project commit	a84b33c - Arabic handoff guide added
Default local URL	http://127.0.0.1:8010
Python	3.11 or newer

Important before you start

The repository contains source code and a sample video, but the large AI model files under models/ are intentionally not on GitHub. Send those files separately to the recipient before they run the video pipeline.

1. What the project does

This is a computer-vision system for Formula-style race footage. It accepts a race video or a car image and creates a structured visual analysis. It does not use official race telemetry.

For video, it detects cars, keeps each car's identity as stable as possible, estimates the team from the livery, watches for damage, and produces heuristic events such as a probable impact or overtake. It exports an annotated MP4 and a result page. For images, it detects vehicle damage and saves an annotated copy.

Core models and tools

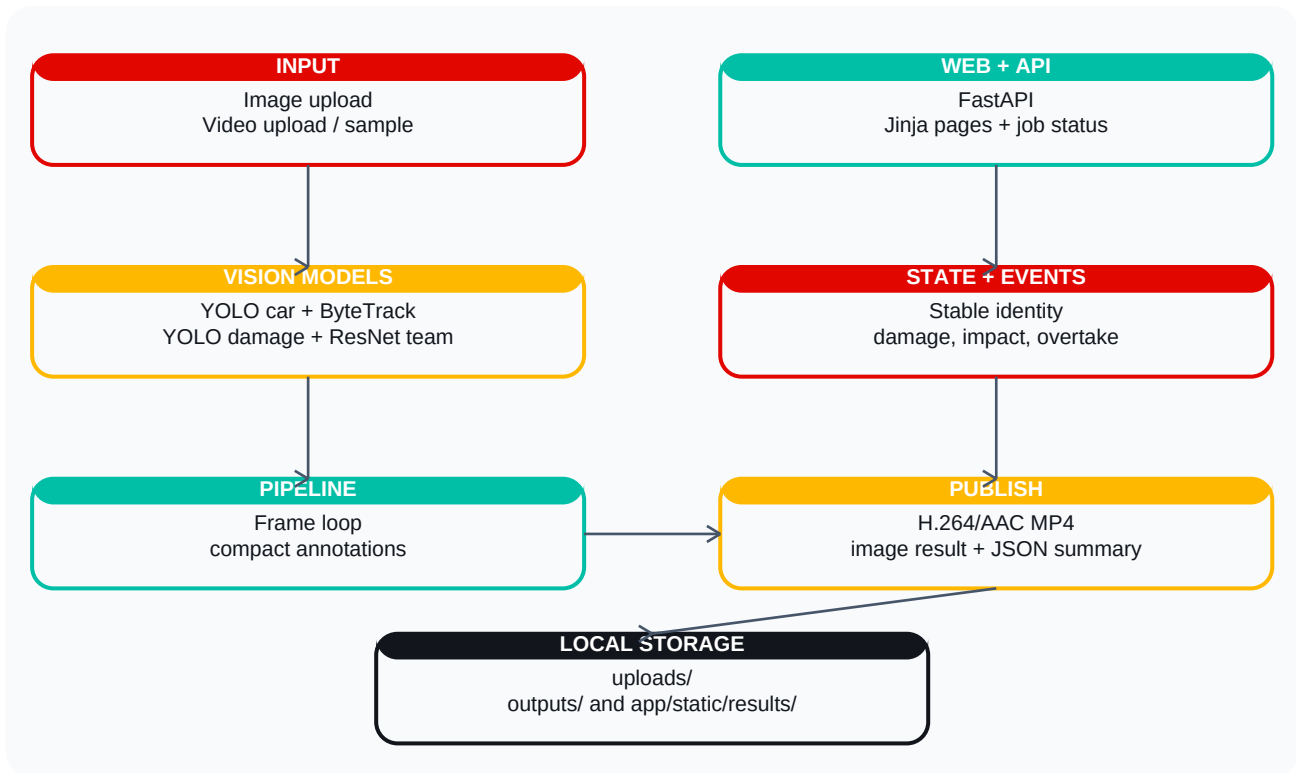
File / folder	What it does and why it exists
YOLOv8 car detector	Detects each race car in every video frame.
ByteTrack	Connects detections across frames and assigns a raw track ID.
YOLOv8 damage detector	Detects dent, scratch, tire flat, and the other CarDD damage classes.
FastAI ResNet34	Classifies the race team from the car livery.
OpenCV + ffmpeg	Reads video, draws overlays, then publishes browser-compatible H.264/AAC video.

Accuracy notes

- Speed is pixels per second, not km/h, because broadcast footage has no GPS or telemetry.
- Collision and overtake results are heuristics to verify against footage, not official race decisions.
- Damage detection needs a clear car crop; very small, distant, or obscured damage can be missed.

2. Architecture

The application is monolithic: FastAPI accepts requests, service wrappers call the computer-vision logic, and `tracking_memory` stores per-car state during one video job. Results are stored locally; there is no database.



Data path: upload or sample video -> vision models -> track and event state -> annotated video and JSON summary -> browser result page.

3. What the recipient must receive

Send the GitHub link and a separate ZIP or cloud folder containing exactly the following files. Do not rename them: the application verifies each model's SHA-256 checksum before loading it.

```
models/  
  best_f1_detect.pt  
  best_carDD.pt  
  f1_team_classifier.pkl
```

These files are excluded by .gitignore because they are large. If they use a different model build, loading will fail with ModelIntegrityError. Send the same files from your machine, or deliberately update the corresponding hash in app/config.py after reviewing the replacement model.

What GitHub already provides

- All Python source, HTML templates, Docker configuration, tests, documentation, and tracker/tracker2.mp4.
- Three small benchmark images in splitted_dataset/val/valid_images/.
- Not current uploads/ or outputs/: the application creates these folders as needed.

Machine requirements

- Windows 10/11, Linux, or macOS. The commands below use Windows PowerShell.
- Python 3.11+. An NVIDIA GPU is optional but makes full video processing faster.
- ffmpeg is required for browser-playable video output. Docker installs it automatically.
- 16 GB RAM or more is recommended when all three models are loaded.

4. Run it from GitHub - Windows PowerShell

Open PowerShell and run these commands in order. Do not open pipeline_result.html directly: it is a Jinja template and must be served through FastAPI.

```
git clone https://github.com/MohammedHssan11/f1-car-vision-intelligence-system.git  
cd f1-car-vision-intelligence-system  
  
py -3.11 -m venv .venv  
.\venv\Scripts\Activate.ps1  
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

Copy the models folder you received into the project root. For example, the final path must be f1-car-vision-intelligence-system\models\best_f1_detect.pt.

```
ffmpeg -version  
py -3.11 -m uvicorn app.main:app --host 127.0.0.1 --port 8010
```

If ffmpeg is missing, install it and reopen PowerShell. On Windows you can use winget install Gyan.FFMpeg. Then open the following URLs:

```
http://127.0.0.1:8010/  
http://127.0.0.1:8010/page/damage  
http://127.0.0.1:8010/page/pipeline  
http://127.0.0.1:8010/docs
```

Port 8010 is the default because port 8000 is often occupied by another FastAPI project. If 8010 is unavailable, use 8011 and update ALLOWED_ORIGINS in .env only if your setup needs it.

5. Easier setup with Docker, plus UI usage

Use this route if Docker Desktop is installed. You still need to copy models/ before the build, because the Dockerfile includes the models in the image.

```
git clone https://github.com/MohammedHssan11/f1-car-vision-intelligence-system.git
cd f1-car-vision-intelligence-system

copy .env.example .env
# Copy models/ here before the next command
docker compose up --build
```

Open <http://localhost:8010> after the build. Stop the service with Ctrl+C and docker compose down when needed.

Quick checks after installation

```
py -3.11 tests\test_reliability.py
py -3.11 tests\smoke_phase1.py
```

test_reliability.py is fast and does not process a complete video. smoke_phase1.py checks API routes, image handling, and the sample video, so it takes longer.

Using the website

- Home (/): shortcuts to image damage analysis, video analysis, and demo actions.
- Damage page (/page/damage): upload JPG, PNG, or BMP. The app saves an annotated image under app/static/results/ and shows a download link.
- Video page (/page/pipeline): upload a video or choose tracker/tracker2.mp4. Use the background job flow and monitor progress on the job page.
- Result page: newly generated videos are H.264/AAC and play in the HTML5 player. Re-run older jobs created before the video-format update.

6. File map - root, app, and templates

File / folder	What it does and why it exists
README.md	Project overview, models, features, performance, and limitations.
requirements.txt	Python dependencies required by the local runtime.
Dockerfile	Builds the Docker image and installs ffmpeg plus Python packages.
docker-compose.yml	Runs the app on host port 8010 and keeps uploads/outputs on the host.
.env.example	Default environment-variable values to copy into .env.
.gitignore / .gitattributes	Excludes models and generated files; handles binary files correctly.
data.yaml	YOLO car dataset definition with one class: f1_car.
cleanup_outputs.py	Removes generated uploads/results after preview, with a --yes option.
app/main.py	Creates FastAPI, configures CORS and static mounts, and registers routes.
app/config.py	Central paths, upload limits, model checksums, and CORS settings.

File / folder	What it does and why it exists
app/api.py	Input validation, endpoints, uploads, template rendering, and result pages.
app/jobs.py	ThreadPoolExecutor background queue and job progress/status store.
app/templates/index.html	Home page.
app/templates/damage_image.html	Damage-image upload and result UI.
app/templates/full_pipeline.html	Video upload/sample selection and background job UI.
app/templates/job_status.html	Progress screen and redirect to the completed result.
app/templates/pipeline_result.html	Final debrief, statistics, browser video player, and downloads.

7. File map - computer vision and tracking

File / folder	What it does and why it exists
services/damage_image.py	Thin wrapper used by app/api.py to call image-damage detection.
services/full_pipeline.py	Thin wrapper used by API/jobs to start the full video pipeline.
src/model_loader.py	Single loading point for YOLO/FastAI, CPU/GPU choice, SHA-256 verification, and lazy cache.
src/detection.py	Image damage detection, confidence filters, and damage-class geometry rules.
src/pipeline.py	Frame loop, tracking, team classification, batched damage checks, annotations, and H.264/AAC publishing.
src/tracking.py	CarState: location, speed, acceleration, smoothed velocity, and history.
src/classification.py	Training script for the ResNet34 team classifier; accepts --dataset-dir and exports to the runtime model path.
tracking_memory/tracker_core.py	Wraps ByteTrack and uses stable-ID recovery to reduce ID switches.
tracking_memory/car_state.py	Bridge to src/tracking.py's CarState.
tracking_memory/damage_state.py	Damage observations, first/last seen, severity, and persistence.
tracking_memory/damage_assigner.py	Associates a damage box with the correct car when IoU is above 0.3.
tracking_memory/collision_detector.py	Emits probable impacts from strong braking plus time-confirmed damage evidence.
tracking_memory/overtake_detector.py	Produces heuristic rank-change events for active cars.
tracking_memory/utils.py	Shared IoU helper.
tracker/bytetrack.yaml	ByteTrack thresholds, matching settings, and lost-track buffer.
tracker/tracker2.mp4	Tracked demo video used for quick pipeline testing.
tracker/tracking_test.py / tracking_test2.py	Legacy visual and headless tracking harnesses.

8. Tests, documentation, execution order

File / folder	What it does and why it exists
tests/test_reliability.py	Fast checks for model integrity, Jinja rendering, ID reassociation, collision timing, and H.264 command/layout.
tests/smoke_phase1.py	Health endpoint, upload validation, image path, and local video pipeline smoke test.
tests/smoke_phase2.py / smoke_phase3.py	Demo results, rendering, background-job progress, failures, and result-page behavior.
tests/smoke_phase4.py	Benchmark-output manifest checks.
benchmarks/	Benchmark samples, manifest, runner, and recorded results.
docs/ARCHITECTURE.md	Architecture drawing and request flow.
docs/AI_SYSTEM.md	Models, thresholds, event logic, and evaluation limits.
docs/BACKEND.md / FRONTEND.md / API_REFERENCE.md	Backend, UI, and HTTP route reference.

File / folder	What it does and why it exists
docs/DEPLOYMENT.md / MAINTENANCE_GUIDE.md / SECURITY_AUDIT.md	Deployment, maintenance, cleanup, tuning, and security guidance.
models/, uploads/, outputs/, app/static/results/	Not source code: model weights or locally generated input/output files.

Video execution sequence

- The user starts a video job from /page/pipeline. app/jobs.py validates the upload and starts one worker.
- src/model_loader.py verifies model checksums and loads weights only on the first request.
- Each frame: YOLO detects cars, ByteTrack creates a raw ID, and TrackIdentityResolver stabilizes it after short occlusions.
- Team classification runs when a car appears; damage is sampled every DAMAGE_EVERY_N_FRAMES and assigned by IoU.
- Collision combines braking and persistent damage evidence. Overtake watches heuristic rank changes.
- OpenCV draws compact labels. ffmpeg converts the intermediate video to H.264/AAC with faststart; the job publishes the result page.

9. Main environment settings and troubleshooting

File / folder	What it does and why it exists
F1_APP_PORT	Docker host port. Default: 8010.
ALLOWED_ORIGINS	CORS allow-list. Default includes localhost:8010.
PIPELINE_CONF_TH / PIPELINE_DAMAGE_CONF_TH / PIPELINE_TEAM_CONF_TH	Minimum confidence values for car, damage, and team predictions.
DAMAGE_EVERY_N_FRAMES	How often damage inference runs. Default: every 10 frames.
MAX_FRAMES	Safety cap for very long videos. Default: 3000.
PIPELINE_H264_CRF	Final video quality/size. Lower is higher quality. Default: 23.
PIPELINE_SHOW_SPEED	Set true to show speed labels. Default false keeps overlays compact.
TRACK_REASSOCIATION_MAX_GAP	Maximum frame gap for stable-ID recovery. Default: 30.
COLLISION_MIN_DAMAGE_OBSERVATIONS	Damage observations required to confirm an impact. Default: 2.

Common problems

File / folder	What it does and why it exists
Model not found	Place all three files in models/ with exactly the expected names.
Integrity check failed	Use the expected model file or review and update the model hash in app/config.py.
ffmpeg is required	Install ffmpeg, reopen PowerShell, and confirm with ffmpeg -version; Docker includes it.
Video player is black	Create a new result. Older mp4v output may not play in Chromium; new jobs use H.264/AAC.
Port already in use	Run on 8011, for example, or change F1_APP_PORT for Docker.

File / folder	What it does and why it exists
Raw {{ }} appears	Do not open templates directly. Use the FastAPI URL only.
Slow processing	Use CUDA GPU, lower MAX_FRAMES, or increase DAMAGE_EVERY_N_FRAMES.
Too many labels	Keep PIPELINE_SHOW_SPEED=false, which is the default compact presentation.

Handoff checklist

- Send the GitHub URL, this PDF, and the complete models/ folder.
- Ask the recipient to clone, install dependencies, copy models/, then start uvicorn.
- Ask them to verify /api/health/ first, then /page/damage and /page/pipeline.
- For a different machine, Docker is often the simplest route because ffmpeg and packages are inside the container.

For deeper work, read README.md, then docs/ARCHITECTURE.md, then src/pipeline.py.